CHAPTER 41

PRACTICAL SPATIAL HASH MAP UPDATES

Pascal Gautron
NVIDIA

ABSTRACT

Spatial hashing is a very efficient approach for storing sparse spatial data in a way that easily allows merging information among areas with similar properties. Though having a very simple core, this technique requires careful implementation to achieve high efficiency. In this chapter we first consider some key principles and implementation aspects. The original spatial hashing is initially limited to storing simple data types, which can be updated using atomic calls. The second part of this chapter adds support for arbitrary data storage in the hash map, while still guaranteeing the atomicity of the updates. We achieve this using on-the-fly generation of change lists for each modified hash map entry.

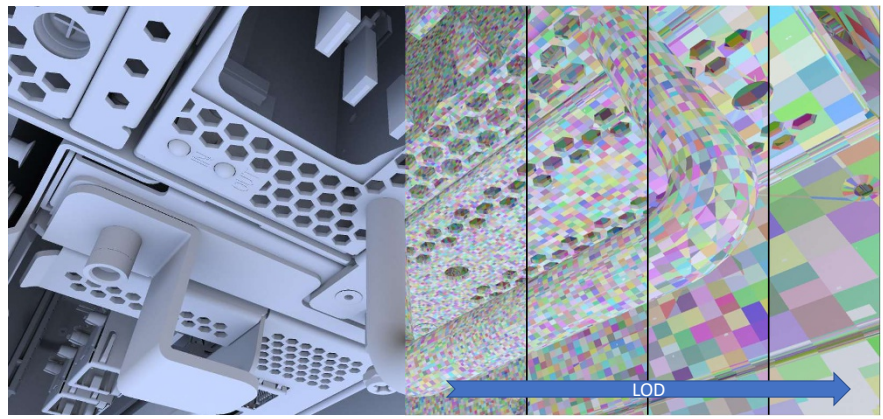

Figure 41-1. *Spatial hashing provides a simple means of storing and retrieving spatial information in constant time, at arbitrary resolutions.*

A. Marrs, P. Shirley, I. Wald (eds.), *Ray Tracing Gems II*, https://doi.org/10.1007/978-1-4842-7185-8_41

41.1 INTRODUCTION

The spatial hashing approach [2] is very efficient to store and access sparse spatial data in a massively parallel setting. This is particularly useful for computing and storing lighting information independently from the scene representation. Using variations on the hash function, a spatial hash map can be extended to support multiresolution and dynamic scenes [4].

Despite its efficiency, spatial hashing suffers from a major limitation on the data stored in the hash map. As many threads may simultaneously insert and modify information in the map, each hash map entry has to be updated atomically. Though this can be trivially achieved for simple data structures, atomicity cannot be guaranteed on more complex data without introducing prohibitive costs.

After introducing key aspects for implementing the spatial hashing scheme, this chapter covers an efficient method for parallel updates of arbitrary data structures within the hash map. Change requests are stored into a set of per-entry linked lists, which are then processed in parallel.

41.2 SPATIAL HASHING

Many light transport algorithms store and interpolate lighting information using a world-space representation such as kD-trees [6], octrees [10], or surface subdivision [5]. However, such data structures usually involve nontrivial representations and update algorithms, making them challenging to use in massively multithreaded contexts. Screen-space methods [1, 3] store information per-pixel for improved efficiency. However, they can only represent a subset of the full 3D scene.

Spatial hashing [2] builds upon the idea of hash maps using a hash function designed so that hashing neighboring world-space points results in the same hash index. In other words, spatial hash entries represent sub-volumes of the scene (Figure 41-1). The hash map can be seen as a sparse voxel representation with trivial data structure and constant-time access.

41.2.1 ENTRY ALLOCATION

The spatial hash function H is based on successively applying a simple, 1D integer hash function h on each coordinate of a 3D point P:

$$H(P) = h\left(\lfloor P_z/d \rfloor + h\left(\lfloor P_y/d \rfloor + h(\lfloor P_x/d \rfloor)\right)\right), \tag{41.1}$$

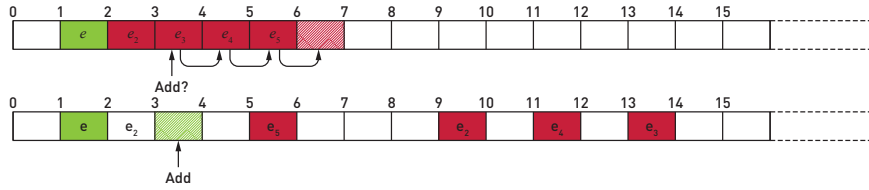

Figure 41-2. *Using linear search, colliding entries mapping to the entry e are stored as e_n in the neighboring entries, hence creating a dense block. Adding another entry in the same area results in many search steps before finding a free slot (top). Using rehashing, the colliding entries are scattered throughout the map, thus reducing the collision rate and preserving the hash map uniformity (bottom).*

where d is a discretization step (i.e., the voxel size). The choice of the integer hash function h is arbitrary, such as the Wang hash [9]. An algorithm can then store light transport data for the related volume at the index $H(P)$ mod s, where s is the number of entries in the hash map.

Collisions are detected by computing and storing a second hash index, or *checksum*, using Equation 41.1 with another integer hash function such as a XOR shift [7]. When adding a new entry into the hash map, we first compute its hash index using the primary hash function. We then generate its checksum and compare it to the checksum stored at the hash index. If the entry is not empty and the checksums differ, a collision has been found. This is conceptually equivalent to using 64-bit hash indices. Please note that though undetected collisions (i.e., same hash and checksum indices) are extremely unlikely, they remain theoretically possible. However, we were not able to isolate such cases in real-world scenarios.

The simplest collision mitigation technique is a linear search scanning the hash map for an empty entry. This technique is cache-friendly, but areas in the hash map corresponding to collisions become very dense, hence introducing further expensive collisions from unrelated areas (Figure 41-2). Rehashing consists in re-applying a hash function to generate another, noncontiguous index (Figure 41-3). While this impacts cache coherence, it also preserves the distribution uniformity and results in overall increased performance in our tests.

41.2.2 REFINING THE HASH FUNCTION

The spatial hash function just described subdivides the scene into equally sized voxels. In a way similar to bilateral filtering [8], the hash function can be

```
/* Entry allocation                                                    */
hashIndex = H_Wang(P)  mod  hashMapEntryCount;
checksum = H_XORShift(P);
searchSteps = 0;
/* Allow a limited number of searches to avoid deadlocks              */
while searchSteps < MAX_SEARCH_STEPS do
    storedChecksum = atomicCAS(Headers[hashIndex], checksum,
     EMPTY);
    if storedChecksum == EMPTY OR storedChecksum == checksum then
        /* The entry at that index was empty or already exists */
        return hashIndex;
    end
    if storedChecksum != checksum then
        /* Collision found, compute another index                 */
        hashIndex = h(hashIndex)  mod  hashMapEntryCount;
        searchSteps++;
    end
end
return NOT_FOUND;
```

Figure 41-3. *Procedure for allocating hash entries and mitigating collisions.*

refined by including additional criteria, such as the surface normal:

$$H_N(P, \mathbf{n}) = h\left(\lfloor n_z.d_N \rfloor + h\left(\lfloor n_y.d_N \rfloor + h\left(\lfloor n_x.d_N \rfloor + H(P)\right)\right)\right), \tag{41.2}$$

where d_N is a discretization constant for the components of the normal vector. Multiple levels of detail (LOD) can also be stored within the hash map by further including an LOD index to the hash function and adapting the spatial discretization. As shown in Figure 41-4, adding those criteria involves allocating additional hash entries to represent the points visible in the final image. Because this also results in less coherent memory accesses, it is crucial to consider the cost-to-benefits ratio of each added criterion.

41.2.3 STORING INFORMATION IN THE HASH MAP

Each hash map entry contains information, also referred to as its *payload* in this chapter. For example, the payload may be a floating-point RGB irradiance value and a counter representing the number of rays used in the irradiance estimate. Because several threads may modify an entry at once (e.g., tracing more rays), the payload is directly altered using atomic functions

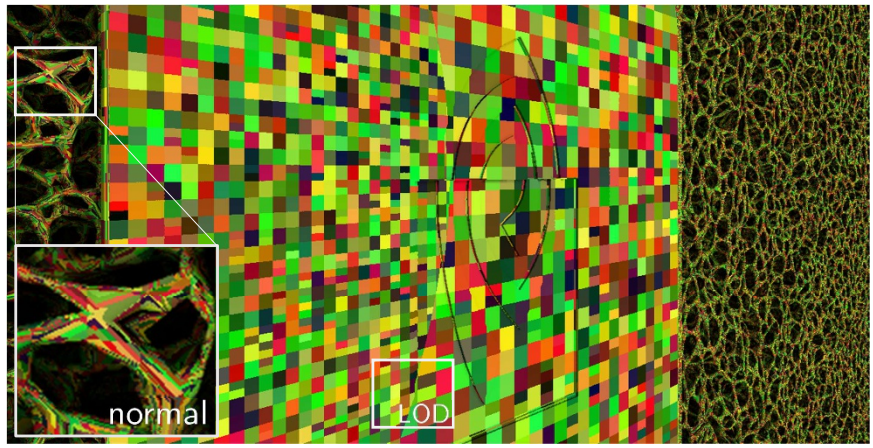

Figure 41-4. *The computation of the hash index (shown in false color) is extended to include normals, hence ensuring points within a hash entry lay on surfaces with similar orientations. The world-space size of the entries are adapted to the viewing distance by adding an LOD index and a discretization size to the hash function as well.*

(Figure 41-5). Note that issuing numerous atomics targeting a single address in global memory may result in a performance loss on graphics hardware. This loss can be particularly significant when the hash map entries cover many pixels in the image, and the payload becomes complex. For further implementation details, we strongly encourage the reader to refer to Binder et al. [2] and Gautron [4].

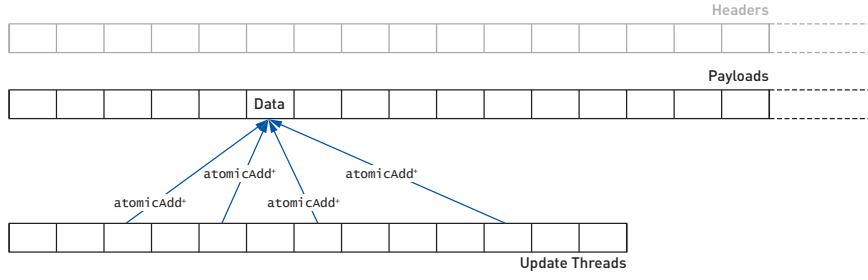

Figure 41-5. *The original spatial hashing algorithm updates hash entries on the fly using atomic functions, which can result in high atomic pressure and data structure restrictions.*

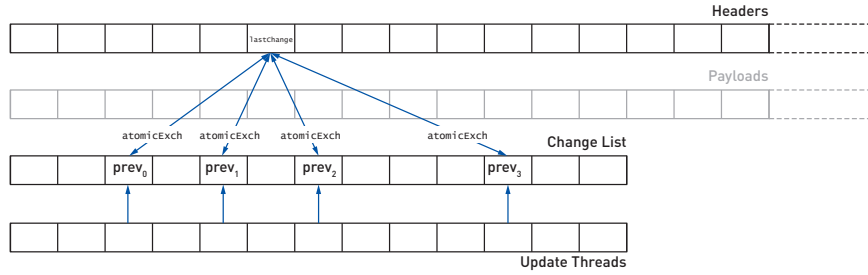

Figure 41-6. *The update step change requests are generated for each update thread, while the hash headers keep track of the index of the last change request touching each entry. Each change request updates that tracker and stores its previous value.*

41.3 COMPLEX DATA STORAGE AND UPDATE

Besides the atomic pressure when using larger payloads, another issue with the direct update is the limitation on the possible data types. For example, though using 16-bit floating-point values would increase cache coherence, neither GLSL for Vulkan nor HLSL provide the corresponding atomic functions. A related issue arises with interdependent components, such as the brightest radiance sample found so far and the index of that sample. A costly mutual exclusion (mutex) would be required to safely update both values.

Instead, we store the changes into a *change list* buffer, where each change is labeled with the index of the hash entry to which it refers. In order to avoid costly sorting operations, we build per-entry change lists on the fly. To this end, we add another unsigned integer `lastChange` to the entry header. When creating the change request *n* in the change list, the algorithm looks up the value `lastChange` of the related entry and atomically exchanges it with the index *n*. The former `lastChange` value is then stored in the `previous` field of the change request. This way, each change request links to the previous change request related to the same entry.

In the example of Figure 41-6, threads 2, 4, 6, and 10 update the same hash entry.[1] Initially we have `lastChange == NO_PRECEDENT`. When thread 2 creates a change request, it atomically exchanges its index with `lastChange`. This sets `lastChange` ← 2 and `previous(2)` ← `NO_PRECEDENT`. Thread 4 performs the same operations, but fetches the updated value of

[1] For clarity we order the operations following the thread numbers. In practice this order is determined by the hardware scheduler.

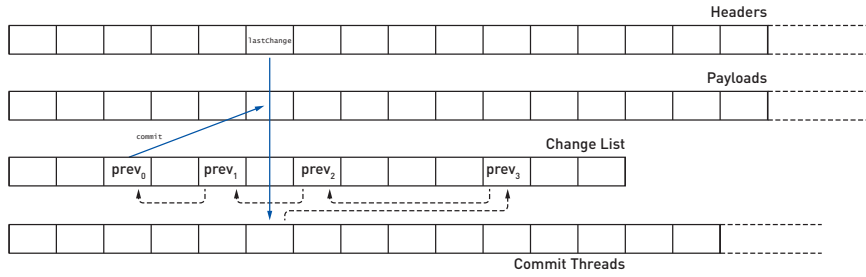

Figure 41-7. *The commit step starts by fetching the index of the last change request for a given hash entry. We then follow the linked list using the indices stored in each of the change requests. After combining the changes, the final update is committed into the hash map.*

`lastChange` == 2. The change request creation yields `lastChange` ← 4 and `previous(4)` ← 2. This is repeated for threads 6 (`lastChange` ← 6, `previous(6)` ← 4) and 10 (`lastChange` ← 10, `previous(10)` ← 6).

At the end of the process, `lastChange` will be the starting point of the commit step, in which the change requests of the modified entries are written into the hash map payloads. This algorithm fetches the change request 10, which links to the previous element in the change list (`previous(10)`== 6), and so on (Figure 41-7). This simple algorithm provides the basis for fast parallel updates of arbitrary hash map payloads. The next section will cover some practical implementation details.

41.4 IMPLEMENTATION

41.4.1 DATA STRUCTURES

The *header* of each hash map entry contains two unsigned 32-bit integers: the collision detection `checksum`, and `lastChange` to link change requests. Because spatial hashing is mainly limited by memory latency, it is crucial to optimize its memory access patterns. In particular, allocating a hash entry potentially requires reading many checksum values to resolve collisions, whereas `lastChange` is only reset once by the thread that allocated the entry. Therefore, the headers are stored in a structure-of-arrays fashion so that all checksums are contiguous. The payloads are stored separately for the same reasons. Their layout, though, depends on their specific data structure and access patterns.

In addition to the hash map, our algorithm uses a *change list* to store the change requests before they are committed. The header of a change request

is an unsigned integer linking to the previous change request for the same entry. This buffer is as large as the largest possible set of simultaneous changes, which is typically the number of pixels in the image. We also keep a buffer representing the unique indices of the hash map entries that have been modified by the change list.

Our algorithm addresses the general case where not all threads may generate a change request, and not all hash entries may be touched at once. We use counters for the change list and the list of touched entries, which are atomically incremented.

41.4.2 HASH ENTRY ALLOCATION

The first step of spatial hashing consists in determining the hash entries corresponding to each shaded point. Once an appropriate entry (i.e., empty or with the same checksum) has been found, we reset the change tracker `lastChange` ← `NO_PRECEDENT`. The change request and touched hash entries counters are also reset to 0.

41.4.3 REQUESTING CHANGES

After generating the information to be stored in the hash map (e.g., computing the incoming radiance value by ray tracing), we reserve an entry in the change list by incrementing its counter. We then fetch the current value of `lastChange` and atomically replace it with the index of the new change request. If this change request is the first one issued for the hash entry (`lastChange == NO_PRECEDENT`), the index of the hash entry is added to the list of touched entries (Figure 41-8). As in the original spatial hashing

```
/* Change generation, typically included in the ray tracing
   kernel                                                    */
foreach Change request r in parallel do
    requestIndex = reserveChangeRequestSlot();
    r.previous = atomicExch(Headers[r.hashEntry].lastChange,
     requestIndex);
    if r.previous == NO_PRECEDENT then
        TouchedList.enqueue(r.hashEntry);
    end
    ChangeList[requestIndex] = r;
end
```

Figure 41-8. *Generation of change requests.*

```
/* Commit the changes to the hash map                                  */
foreach Entry index c in TouchedList in parallel do
    clIndex = Headers[c].lastChange;
    totalChange = emptyContribution();
    while clIndex != NO_PRECEDENT do
        totalChange = merge(totalChange, ChangeList[clIndex]);
        clIndex = Headers[c].lastChange;
    end
    Payloads[c] = merge(Payloads[c], totalChange);
    /* Propagate the combined change to the next coarser LOD  */
    if hasParentLOD(c) then
        parentLODEntry = getParentLODEntryIndex(c);
        parentChangeRequest = createChangeRequest(parentLODEntry,
          totalChange);
        Reapply algorithm with parentChangeRequest;
    end
end
```

Figure 41-9. *Final write of the change list in the hash map, with propagation to coarser LODs.*

scheme, several threads may modify the value of `lastChange` in a given entry simultaneously. However, our change list–based approach uses a single atomic exchange call regardless of the payload size, making it scale better to more complex data.

41.4.4 COMMITTING CHANGE REQUESTS

The header of each touched entry links to the last related change request, which, in turn, indirectly links to all other change requests for that entry. Ideally, the contributions of all the change requests can be merged into one, which is then committed into the hash map. This reduces the global memory traffic by keeping most of the working set in local memory (Figure 41-9). Otherwise, the changes would have to be serially written into the hash map, which introduces an additional cost.

41.4.5 PROPAGATING TO COARSER LODS

Spatial hashing can implicitly store multiresolution data (Figure 41-1) by simply adding a level of detail index into the hash function [2]. Further efficiency can be obtained by propagating information from finer LODs to coarser LODs and by leveraging this approach to locally find a trade-off between noise and spatial accuracy [4]. The payload at the coarser LODs is

obtained as a byproduct of the sampling at the finer LOD: each time new samples are traced, their contributions are also written into the coarser LODs.

Because using a coarse LOD means touching fewer entries that cover many pixels, generating one change request per pixel results in only a few threads processing very long change lists. This, in turn, results in poor utilization on modern graphics hardware running thousands of threads in parallel. However, in most cases change requests can be combined into one. As indicated in the previous section, this merging ability is a key to the efficiency of the update mechanism. Once a change list has been committed to its finer-LOD entry, we create a single, *combined* change request for the same location at a coarser LOD.

This technique requires allocating a second change list where those changes are recorded. It also introduces two additional passes: a cleanup pass clears the change trackers for all touched entries, and a propagation pass builds the list of touched entries from the new change list. The committing pass can then be applied to the newly generated change list. This process can then be repeated to propagate to further, coarser LODs if necessary.

41.5 APPLICATIONS

We applied these techniques for real-time computation of ray traced shadows, applied to ambient occlusion and environment lighting. Our implementation uses the Vulkan API, running on a GeForce RTX 3090 to generate images at resolution 1920×1080.

41.5.1 AMBIENT OCCLUSION

Ambient occlusion (AO) is a very useful approximation of global illumination, where the ambient term is modulated based on the proximity of occluding geometry. In this case the payload of each hash entry contains the number of samples traced so far and the number of actual occlusions found (Figure 41-10).

```
struct Payload contains
    uint16_t occlusions;
    uint16_t sampleCount;
end
```

Figure 41-10. *For ambient occlusion each hash entry fits 32 bits to store occlusion and sample counters.*

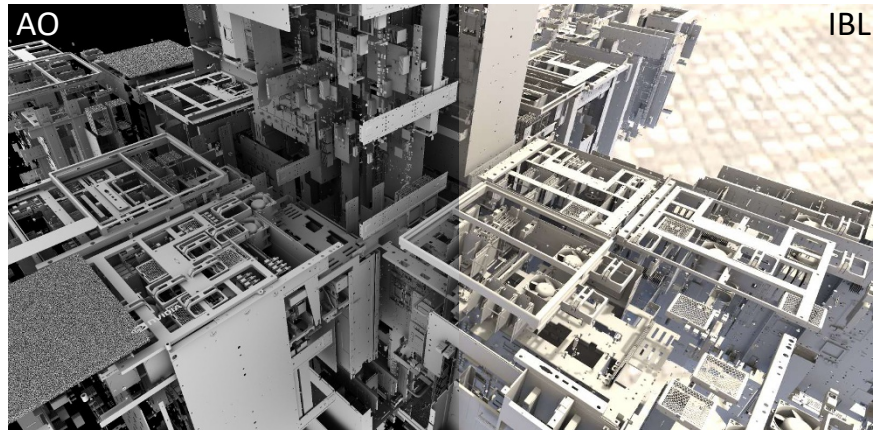

Figure 41-11. *The performance of our update scheme is on par with the atomics-based update using simple data types such as for ambient occlusion (left). It also expands the storage capabilities of the spatial hash map to support more complex payloads such as 16-bit floating-point data (image-based lighting, right).*

Due to its simplicity, this payload structure can be updated directly using a single unsigned integer `atomicAdd` operation, provided the sample count does not exceed 65,535. We compared the direct update with our approach in a CAD scene comprising 304 million triangles (Figure 41-11). All other aspects of the AO computation algorithm being equal, the overall render time per frame for both techniques is equivalent: 7.04 ms for direct updates versus 7.01 ms with our change list–based technique. Though the list management introduces some overhead, this is compensated by the reduced atomic pressure in the LOD propagation. Our algorithm has been integrated in the real-time viewport of the industry-standard Dassault Systèmes 3DEXPERIENCE platform (Figure 41-12).

41.5.2 ENVIRONMENT LIGHTING

This application considers the computation of shadows cast by a spherical environment. The payload of a hash map entry is the accumulated irradiance value at the corresponding location. In order to reduce the memory footprint of the hash map, the payload is defined on 64 bits (Figure 41-13).

Combined with a 64-bit header for each hash entry, and with the typical five million entries in the hash map, the subsequent 128-bit entries result in a memory occupancy of 80 MB. Atomic operations on `f16vec3` types are not supported in GLSL at this time, making our solution the only option for efficient updates.

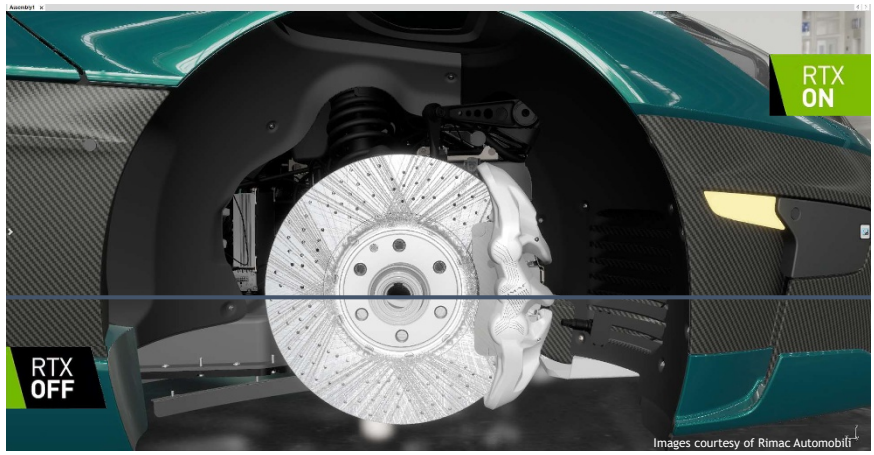

Figure 41-12. *Our algorithm (top) has been integrated within the Dassault Systèmes 3DEXPERIENCE platform as an alternative to SSAO (bottom). The rendered model contains 50 million triangles. (Model courtesy of Rimac Automobili.)*

```
struct Payload contains
    f16vec3 rgb;
    uint16_t sampleCount;
end
```

Figure 41-13. *Environment lighting requires accumulating floating-point values, stored on 16 bits for performance.*

41.6 CONCLUSION

Spatial hashing is a GPU-friendly method for representing sparse data in world space. Despite its efficiency, it originally suffers from limited representation capabilities due to the need of atomic functions to update the contents of the hash map. Using a single temporary variable and a deferred update mechanism, the update technique of this chapter enables dynamic storage and updating of complex data by generating per-entry linked lists. The resulting technique is applied to ray traced ambient occlusion and image-based lighting in massive CAD scenes. This technology has been integrated into the real-time viewport of Dassault Systèmes Catia.

As using hash maps to represent world-space data is relatively new, it carries a strong inspirational value for future work. In particular, the optimization of

the hash function for improved cache coherence holds vast potential for performance optimization. Extensions for nearest-neighbor searches and spatial filtering will also extend the domains in which spatial hashing can be applied.

REFERENCES

[1] Bavoil, L., Sainz, M., and Dimitrov, R. Image-space horizon-based ambient occlusion. In *ACM SIGGRAPH 2008 Talks*, 22:1, 2008. DOI: 10.1145/1401032.1401061.

[2] Binder, N., Fricke, S., and Keller, A. Fast path space filtering by jittered spatial hashing. In *ACM SIGGRAPH 2018 Talks*, 71:1–71:2, 2018. DOI: 10.1145/3214745.3214806.

[3] Bitterli, B., Wyman, C., Pharr, M., Shirley, P., Lefohn, A., and Jarosz, W. Spatiotemporal reservoir resampling for real-time ray tracing with dynamic direct lighting. *ACM Transactions of Graphics (Proceedings of SIGGRAPH)*, 39(4):148:1–148:17, 2020. DOI: 10.1145/3386569.3392481.

[4] Gautron, P. Real-time ray-traced ambient occlusion of complex scenes using spatial hashing. In *ACM SIGGRAPH 2020 Talks*, 5:1–5:2, 2020. DOI: 10.1145/3388767.3407375.

[5] Goral, C. M., Torrance, K. E., Greenberg, D. P., and Battaile, B. Modeling the interaction of light between diffuse surfaces. In *SIGGRAPH '84: Proceedings of the 11th Annual Conference on Computer Graphics and Interactive Techniques*, pages 213–222, 1984. DOI: 10.1145/800031.808601.

[6] Jensen, H. W. *Realistic Image Synthesis Using Photon Mapping*. A K Peters, 2001.

[7] Marsaglia, G. XORShift RNGs. *Journal of Statistical Software*, 008(i14), 2003. https://EconPapers.repec.org/RePEc:jss:jstsof:v:008:i14.

[8] Tomasi, C. and Manduchi, R. Bilateral filtering for gray and color images. In *Sixth International Conference on Computer Vision*, pages 839–846, 1998. DOI: 10.1109/ICCV.1998.710815.

[9] Wang, T. Integer hash function. https://gist.github.com/badboy/6267743, 1997.

[10] Ward, G., Rubinstein, F., and Clear, R. A ray tracing solution for diffuse interreflection. In *SIGGRAPH '88: Proceedings of the 15th Annual Conference on Computer Graphics and Interactive Techniques*, pages 85–92, 1988. DOI: http://doi.acm.org/10.1145/54852.378490.